

I. Use Case Description	
Use Case Name	<i>Ontology-Guided Agentic Tool and Skill Orchestration</i>
Use Case Identifier	<i>ATSO-001</i>
Source	<i>CSCI 6340 - Ontologies</i>
Point of Contact	<i>Gia Oriana Santos (santog@rpi.edu)</i>
Creation / Revision Date	<i>9/1/2026</i>
Associated Documents	

II. Use Case Summary	
Goal	<i>To govern which agent skills and tools to invoke, as well as determine the most optimal orchestration plan an LLM should follow given the user's request and constraints</i>
Requirements	<i>This use case must support the following capabilities</i> • <i>Skill and tool discovery</i> • <i>Orchestration plan generation</i> • <i>Dependency checking with skills and tools</i> • <i>Constraint-Aware orchestration plan selection</i> • <i>Explanability and provenance of selected orchestration plan</i> • <i>Parallel Execution Planning</i>
Scope	• <i>LLM agent retrieves scholarly articles and answers questions for a specific domain</i> • <i>External servers available to the LLM</i> ◦ <i>PubMed MCP</i> ◦ <i>Supplement Advisor MCP</i> ◦ <i>OpenFDA MCP</i> ◦ <i>iHerb-CLI</i> • <i>Specific domain: personalized supplement recommendation</i> • <i>Not considering "common knowledge" questions. Only questions related to personalized supplement recommendations</i>
Priority	<i>N/A</i>
Stakeholders	• <i>End users</i> • <i>LLM agent developers</i> • <i>External server providers</i>
Description	<i>The emergence of the MCP has enabled LLMs to access external tools and data sources through standardized interfaces, allowing AI agents to perform complex retrieval, reasoning, and decision-making tasks. As AI agents become increasingly modular, they must not only determine which external servers and tools to invoke, but also decide when specialized agentic skills should be executed to accomplish subtasks for an overall goal. As the number of available skills and tools continues to grow, LLM agents face increasingly difficult orchestration problems, including determining which skills and tools should be executed, dependencies between skills and tools in orchestration, choosing the best valid orchestration plan given the particular user and situation, and explaining why the orchestration plan should be preferred over another. Current LLMs typically make these decisions implicitly through prompting or learned behavior, making the orchestration process difficult to explain, reproduce, validate, and optimize.</i> <i>This use case addresses the ambiguity of agentic skill and tool orchestration by incorporating an ontology as a layer between the user and agent to explicitly represent the capabilities, inputs, outputs, dependencies, execution requirements, and characteristics of available skills, external servers, and their associated tools. Given a user's request, the ontology guides the LLM</i>

	agent in selecting the appropriate skills, identifying the relevant external servers and tools, determining the optimal sequence of skill executions and tool invocations, and selecting the orchestration plan that best satisfies the task requirements. The use case's scope focuses on generating evidence-based, personalized supplement recommendations by integrating information from a user's health profile and preferred execution settings to act as constraints on orchestration plan selection. This use case also incorporates external servers to support retrieval from scientific literature, safety screening, and an existing product catalog. By grounding orchestration decisions in explicit ontological knowledge, the framework enables transparent, explainable, reusable, and repeatable agent workflows while ensuring that supplement recommendations are supported by user-specified constraints for semantic orchestration plan generation
Actors / Interfaces	Primary actors • User • LLM Agent Secondary actors (resources) • PubMed MCP server • Supplement Advisor server • OpenFDA server • iHerb-CLI
Pre-conditions	• Ontology is loaded into the system • LLM agent model must be able to skills and tools (including MCP and CLI). • LLM agent must have the following skills ◦ Supplement Recommendation ◦ Safety Screening ◦ Evidence Verification ◦ Product Selection • LLM agent is connected to the following external servers ◦ PubMed MCP ◦ Supplement Advisor MCP ◦ OpenFDA MCP ◦ iHerb-CLI • LLM agents must also have access to the following to facilitate constraint-aware selection ◦ User health profile ◦ User's execution settings • User inputs a request to the LLM agent
Post-conditions	• LLM agent provides an answer to the user's request
Triggers	• User sends a request to the LLM agent.
Performance Requirements	
Assumptions	
Open Issues	

III. Usage Scenarios

Provide at least two usage scenarios that flesh out the requirements outlined in the summary, including identification of requirements specific to any envisioned ontology or semantically-driven service or application. Scenarios should be described as narrative, with supporting diagrams as appropriate. In an Agile process, every user story relevant to the use case should be included and elaborated/rolled up into one or more usage scenarios, with a clear mapping from the user story to the scenario it is integrated in or mapped to.

1. Ken is a 21-year-old male college student who exercises 5–6 days per week, combining resistance training with cardiovascular training as he prepares to become a hybrid athlete. His primary goals are to build lean muscle while supporting long-term health, particularly bone strength and cellular support, so he can maintain an active lifestyle for years to come. He follows a Mediterranean diet, has no known medical conditions or injuries, and does not take any prescription medications.

Ken uses the Supplement Recommender through his LLM. Although he has a paid LLM subscription, he prefers to minimize token usage and external tool calls whenever possible. As a college student on a budget, he is looking for affordable supplements with strong customer reviews and good value. He wants recommendations that are supported by scientific evidence but is willing to accept slightly less comprehensive analysis if it reduces cost. He also wants to avoid purchasing unnecessary or overlapping supplements that provide redundant benefits.

2. Jessica is a woman in her late 60s with osteoarthritis who has begun experiencing age-related declines in bone density. She remains active through low-impact activities such as walking, yoga, and tai chi but avoids strenuous exercise because of joint pain. She takes 200 mg of celecoxib (Celebrex) daily to manage her arthritis symptoms. Jessica's primary goal is to improve her bone health while supporting overall longevity, including maintaining cardiovascular health, mobility, and healthy aging.

She uses the Supplement Recommender through her LLM and also has a paid LLM subscription. She is not concerned about the cost of LLM usage or supplement prices. Instead, she prioritizes recommendations that are supported by high-quality scientific evidence and thoroughly evaluated for safety. She wants high-quality supplements with minimal risk of adverse effects and expects the system to screen for potential interactions with her prescription medication before making any recommendations.

IV. Basic Flow of Events

Narrative: Often referred to as the primary scenario or course of events, the basic flow defines the process/data/work flow that would be followed if the use case were to follow its main plot from start to end. Error states or alternate states that might occur as a matter of course in fulfilling the use case should be included under Alternate Flow of Events, below. The basic flow should provide any reviewer a quick overview of how an implementation is intended to work. A summary paragraph should be included that provides such an overview (which can include lists, conversational analysis that captures stakeholder interview information, etc.), followed by more detail expressed via the table structure.

In cases where the user scenarios are sufficiently different from one another, it may be helpful to describe the flow for each scenario independently, and then merge them together in a composite flow.

Basic / Normal Flow of Events			
Step	Actor (Person)	Actor (System)	Description
1	user		Submits a request to the agent
2	-	system	Parses request, extracting goals, constraints, and user preferences
3	-	system	Retrieves user's health profile
4	-	System (KG)	Identifies information requirements from parsed user request and health profile
5	-	System (KG)	Identifies candidate agent tools and skills from the information requirements
6	-	System (KG)	Determines candidate orchestration plans based on a

			combination of required agent tools and skills that satisfy the information requirements
7	-	System (KG)	Evaluates candidate orchestration plans by applying constraints (user constraints and planning constraints) and ranking them based off various decision criteria
8	-	System (KG)	Selects orchestration plan
9	-	system	Derives orchestration dependencies from the input/outputs of orchestration steps into a dependency graph
10	agent	-	Executes orchestration plan by invoking agent tools and skills according to the plan
11	agent		Generates answer to user's request
12		System (KG)	Records skill and tool invocations from the orchestration as well as decision criteria as provenance

V. Alternate Flow of Events

Narrative: The alternate flow defines the process/data/work flow that would be followed if the use case enters an error or alternate state from the basic flow defined, above. A summary paragraph should be included that provides an overview of each alternate flow, followed by more detail expressed via the table structure.

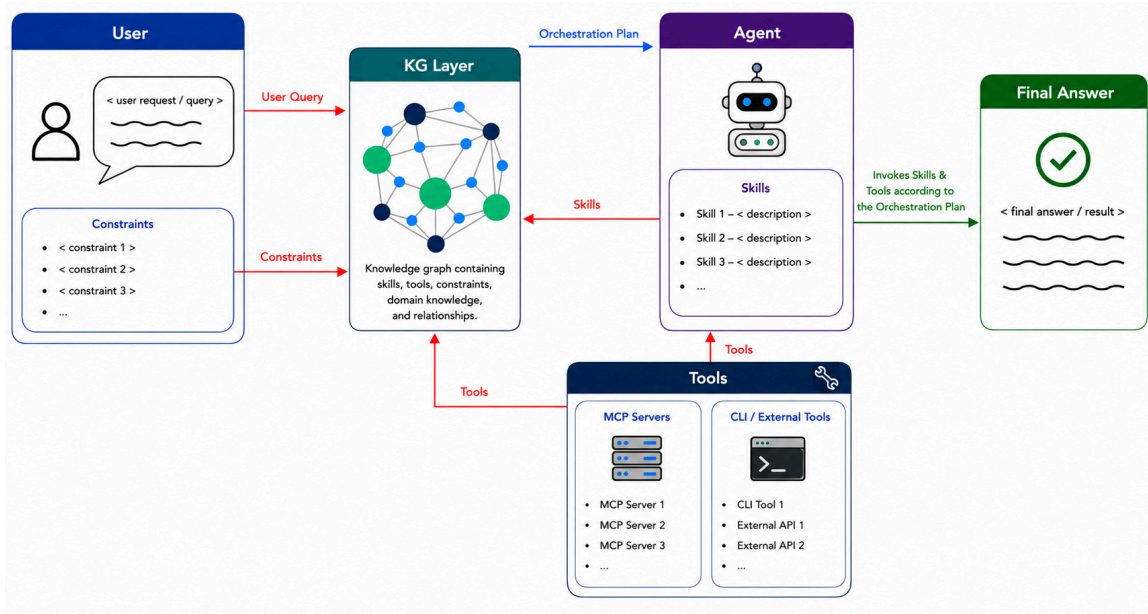
Alternate Flow of Events: Additional Safety Requirement is Inferred			
Step	Actor (Person)	Actor (System)	Description
1	user		Submits a request to the agent
2	-	system	Parses request, extracting goals, constraints, and user preferences
3	-	system	Retrieves user's health profile
4	-	System (KG)	Identifies information requirements from parsed user request and health profile
5		system	Detects medication use from the user's health profile
6		System (KG)	Infers additional requirement to include safety screening into the orchestration plan
7	-	System (KG)	Identifies candidate agent tools and skills from the information requirements
8	-	System (KG)	Determines candidate orchestration plans based on a combination of required agent tools and skills that satisfy the information requirements
9	-	System (KG)	Evaluates candidate orchestration plans by applying constraints (user constraints and planning constraints) and ranking them based off various decision criteria
10	-	System (KG)	Selects orchestration plan
11	-	system	Derives orchestration dependencies from the input/outputs of orchestration steps into a dependency graph
12	agent	-	Executes orchestration plan by invoking agent tools and skills according to the plan
13	agent		Generates answer to user's request
14	-	System (KG)	Records skill and tool invocations from the orchestration as well as decision criteria as provenance

VI. Use Case and Activity Diagram(s)

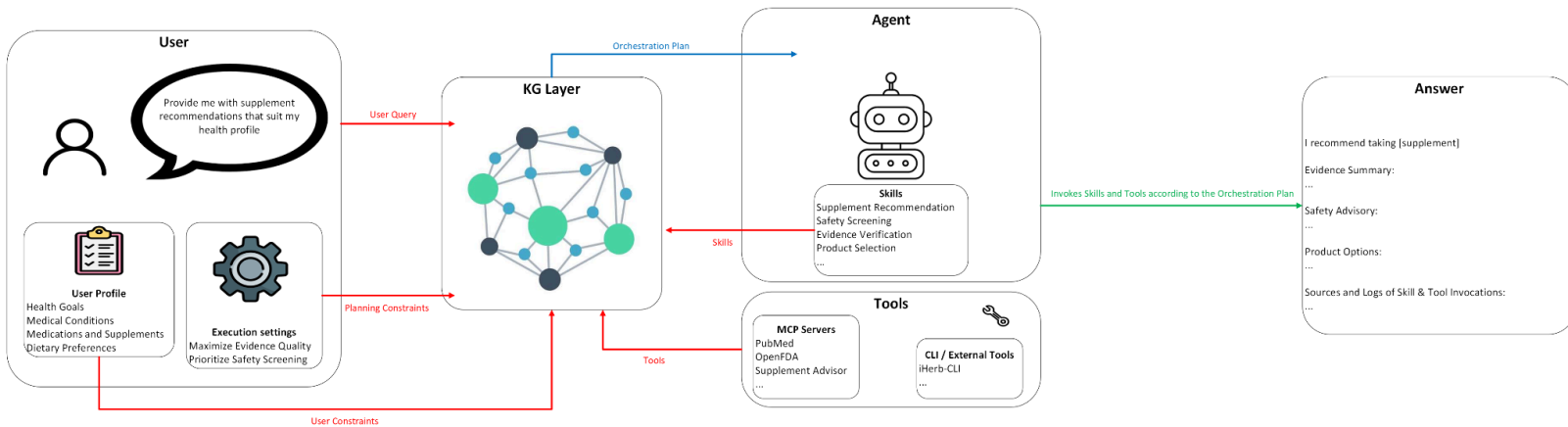
Provide the primary use case diagram, including actors, and a high-level activity diagram to show the flow of primary events that include/surround the use case. Subordinate diagrams that map the flow for each usage scenario should be included as appropriate

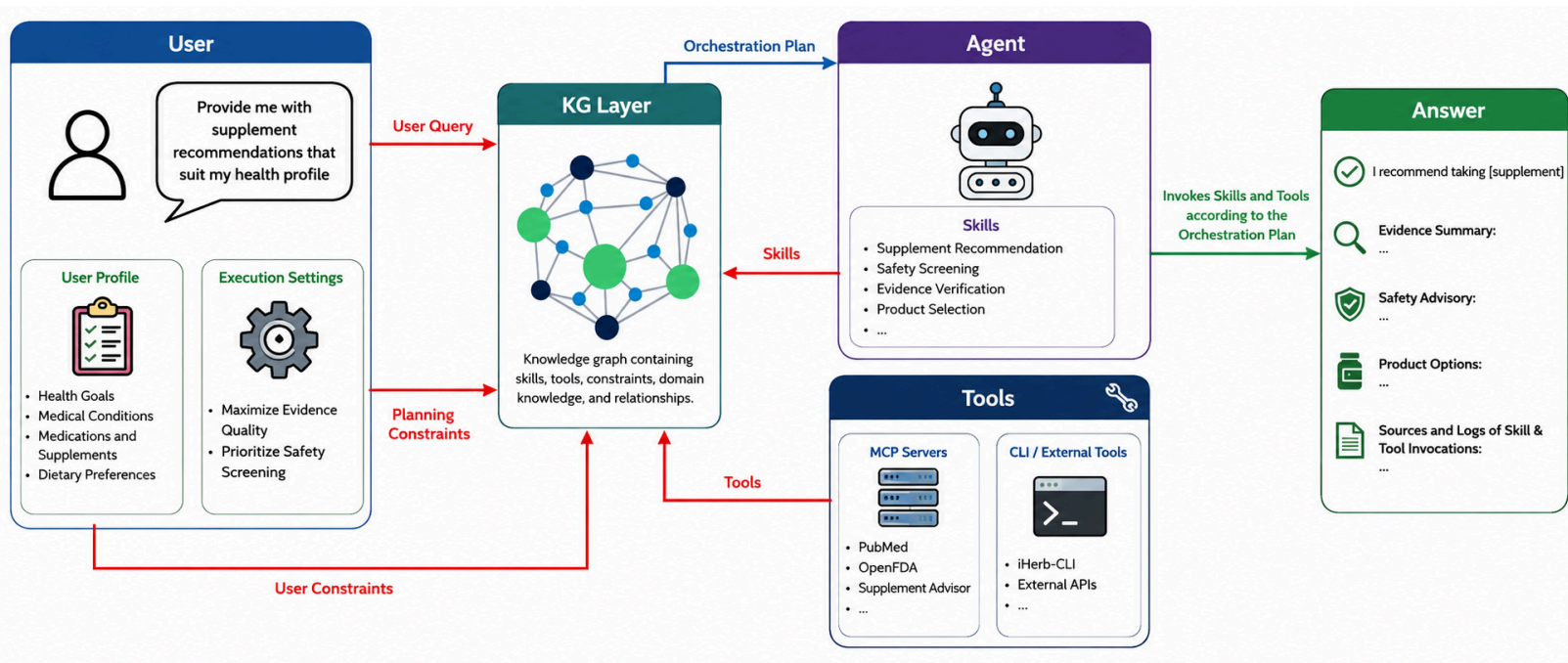
Concept of Operations

For a general ontology-guided agentic orchestration use case



For a specific scope: personalized supplement recommendation





VII. Competency Questions

Provide at least 2 competency questions that you will ask of the vocabulary/ontology/knowledge base to implement this use case, including example answers to the questions.

Describe at least one way you expect to use the semantics and/or provenance to propose an answer to the questions. Include an initial description of why the semantics and/or provenance representation and reasoning provides an advantage over other obvious approaches to the problem. (optional – depending on the use case and need for supporting business case).

1. What external servers are need to be called to collectively satisfy all information requirements needed to answer the user's request?

Ex. provide me with supplement recommendations that suits my health profile

Usage Scenario 1 (Ken) Answers:

- **Supplement Advisor MCP**
 - Identifies candidate supplements based on the user's health goals, medical conditions, and medical interactions with other supplements and medications the user takes
 - Provides initial set of supplement recommendations before downstream analysis
- **PubMed MCP**
 - Retrieves peer-reviewed biomedical evidence to support the effectiveness of the recommended supplements
 - 2nd priority but still useful in Ken's case
- **iHerb-CLI**
 - Retrieves live supplement products with prices
 - Useful in Ken's case since his top priority is affordable supplements

Usage Scenario 2 (Jessica) Answers:

- **Supplement Advisor MCP**
 - Identifies candidate supplements based on the user's health goals, medical conditions, and medical interactions with other supplements and medications the

- **user takes**
 - **Provides initial set of supplement recommendations before downstream analysis**
- **PubMed MCP**
 - **Retrieves peer-reviewed biomedical evidence to support the effectiveness of the recommended supplements**
 - **Top priority: supplements backed up by evidence**
- **Open FDA MCP**
 - **Screens supplements for potential drug interactions, adverse effects, contraindications, recalls, and other regulatory safety information**
 - **Required since Jessica is taking medication or has medical conditions that could introduce safety concern**
- **iHerb-CLI**
 - **Retrieves live supplement products with ratings**
 - **Useful in Jessica's case since she wants high-quality supplements**

Why an ontology would be useful

- underlying information requirements implied by the user's request (scientific evidence retrieval, medication interaction, safety screening)
- Maps information requirements to external servers capable of satisfying them
 - Supplement Advisor MCP => personalized supplement recommendations
 - PubMed MCP => Evidence-based biomedical literature
 - OpenFDA => drug interactions, adverse events, contraindications, recalls
 - iHerb-CLI => live supplement product prices and ratings
- Reasons over whether certain information requirements are necessary
 - User not taking medication may not require safety screening
 - Users taking medication require additional safety screening check
- Avoids unnecessary tool invocations

How to use the ontology

1. System extracts the user's health profile for their health goals, medications, dietary preferences, and other information required from the query
2. System queries the ontology-grounded KG to identify which external servers satisfy each information requirement (ie. supplement recommendation => Supplement Advisor, evidence retrieval => PubMed)
3. Ontology reasons over constraints to determine whether additional information requirements are implied by the user's profile (ex. Jessica's current medication implies her agent orchestration needs to perform safety screening)
4. If multiple external server combinations satisfy the user's request, the ontology evaluates them according to the modeled decision criteria (ie., information coverage, domain authority, execution cost, etc.)
 - a. Ex. Data from Supplement Advisor is grounded on PubMed but the evidence is static. PubMed MCP enables dynamic biomedical literature retrieval. PubMed has higher information coverage and domain authority compared to Supplement Advisor that its server should be invoked if evidence retrieval is one of the information requirements, even if Ken wants to minimize execution cost.
5. The system selects the minimal set of external servers that collectively satisfies all information requirements while satisfying constraints
6. System records decision criterion as provenance in the KG

2. What execution orchestration should the agent follow to answer a user's request?

Ex. provide me with supplement recommendations that suit my health profile

Candidate Answers:

Usage scenario 1 (Ken) orchestration plan:

1. Get user profile

2. **Get candidate supplements**
 - a. **Supplement_advisor: recommend_supplement(search_query) => supplement recommendations**
3. **Remove redundant supplements**
4. **Get supplement products**
 - a. **iHerb-CLI: search(supplement, sort=[price]) => list of supplement products**
5. **Rank supplement products by cost**
6. **Verify evidence**
 - a. **PubMed: search_articles(search_query) => PMIDs**
 - b. **PubMed: download_articles_batch(PMIDs) => article abstracts**
 - c. **summarize_evidence()**
7. **Rank supplement products by evidence**
8. **Select supplement product**
9. **Produce recommendations**

Usage scenario 2 (Jessica) orchestration plan:

1. **Get user profile**
2. **Get candidate supplements**
 - a. **Supplement_advisor: recommend_supplement() => supplement recommendations**
3. **Screen interactions**
 - a. **OpenFDA: search_drug_adverse_events(medication/supplement)**
 - b. **OpenFDA: search_drug_labels(supplement)**
 - c. **OpenFDA: search_drug_recalls(supplement)**
4. **Rank by screening**
5. **Verify evidence**
 - a. **PubMed: search_articles(search_query) => PMIDs**
 - b. **PubMed: download_articles_batch(PMIDs) => article abstracts**
 - c. **summarize_evidence()**
6. **Rank by evidence**
7. **Get supplement products**
 - a. **iHerb-CLI: search(supplement, sort=[rating]) => list of supplement products**
8. **Rank supplement products by rating**
9. **Select supplement product**
10. **Produce recommendations**

How to solve with ontology

1. System parses the user's request and retrieves any constraints and information requirements from the request
2. System extracts the user's profile, including their health profile for their health goals, medications, dietary preferences as well as optimization constraints (ie., high-quality evidence, reduce supplement cost, minimize execution cost, maximize safety screening, etc)
3. System queries the ontology-grounded KG to identify which external servers and skills satisfy each information requirement
4. Ontology reasons over tool and skill dependencies, modeled by their expected inputs and outputs
 - a. **recommend_supplement(health profile) => candidate supplement**
 - b. **PubMed:search_articles(candidate_supplement) => PMID**
 - c. **PubMed:download_articles(PMID) => article abstract**
5. System infers execution order based on the dependencies modeled by the KG
6. System creates a workflow graph as a DAG based on the inferred execution order and is stored under the class Orchestration Plan in the ontology-grounded KG
7. The system selects that orchestration plan for the agent to follow

3. Which workflow steps of a given orchestration should be executed in parallel while satisfying both workflow dependencies and the user's execution preferences?

Usage scenario 1 (Ken) orchestration plan:

1. *Get user profile*
2. *Get candidate supplements*
 - a. *Supplement_advisor: recommend_supplement(search_query) => supplement recommendations*
3. *Remove redundant supplements*
4. *Get supplement products*
 - a. *iHerb-CLI: search(supplement, sort=[price]) => list of supplement products*
5. *Rank supplement products by cost*
6. *Verify evidence*
 - a. *PubMed: search_articles(search_query) => PMIDs*
 - b. *PubMed: download_articles_batch(PMIDs) => article abstracts*
 - c. *summarize_evidence()*
7. *Rank supplement products by evidence*
8. *Select supplement product*
9. *Produce recommendations*

Candidate Answer:

- **Get supplement products**
 - **iHerb-CLI: search(supplement, sort=[price]) => list of supplement products**
- **Verify evidence**
 - **PubMed: search_articles(search_query) => PMIDs**
 - **PubMed: download_articles_batch(PMIDs) => article abstracts**
 - **summarize_evidence()**

Usage scenario 2 (Jessica) orchestration plan:

1. *Get user profile*
2. *Get candidate supplements*
 - a. *Supplement_advisor: recommend_supplement() => supplement recommendations*
3. *Screen interactions*
 - a. *OpenFDA: search_drug_adverse_events(medication/supplement)*
 - b. *OpenFDA: search_drug_labels(supplement)*
 - c. *OpenFDA: search_drug_recalls(supplement)*
4. *Rank by screening*
5. *Verify evidence*
 - a. *PubMed: search_articles(search_query) => PMIDs*
 - b. *PubMed: download_articles_batch(PMIDs) => article abstracts*
 - c. *summarize_evidence()*
6. *Rank by evidence*
7. *Get supplement products*
 - a. *iHerb-CLI: search(supplement, sort=[rating]) => list of supplement products*
8. *Rank supplement products by rating*
9. *Select supplement product*
10. *Produce recommendations*

Candidate Answer:

- **Screen interactions**
 - **OpenFDA: search_drug_adverse_events(medication/supplement)**
 - **OpenFDA: search_drug_labels(supplement)**
 - **OpenFDA: search_drug_recalls(supplement)**
- **Get supplement products**
 - **iHerb-CLI: search(supplement, sort=[price]) => list of supplement products**
- **Verify evidence**
 - **PubMed: search_articles(search_query) => PMIDs**
 - **PubMed: download_articles_batch(PMIDs) => article abstracts**
 - **summarize_evidence()**

How to solve with ontology:

1. The system retrieves the user's **Orchestration Plan** from the ontology-grounded knowledge graph.
2. The system constructs a workflow dependency graph by reasoning over the semantic relationships between workflow steps. Each **Skill** and Tool Invocation is linked to the Information Artifacts it consumes and produces (e.g., **HealthProfile**, **CandidateSupplement**, **PMID**, **ArticleAbstract**, **ProductList**, **SafetyAssessment**).
3. The ontology infers dependency relationships between workflow steps. A workflow step is dependent on another if it consumes an **Information Artifact** produced by that step.
4. The system identifies independent workflow branches by finding workflow steps whose required **Information Artifacts** have already been satisfied and that do not consume outputs produced by one another.
5. The system returns the set of workflow steps eligible for concurrent execution, allowing the agent to execute those steps in parallel while preserving all semantic dependencies.

4. Why did the agent recommend supplement X instead of supplement Y?

Usage scenario 2 (Jessica): Vitamin D3 > Calcium Carbonate

Candidate Answers:

- **PubMed identified stronger evidence supporting vitamin D3 supplementation that fit Jessica's profile (older adult, improving bone health)**
 - Vitamin D3 helps older adults absorb calcium for stronger bones
 - Normal levels of vitamin D3 are generally safe and show little contraindications with other medications
- **PubMed identified evidence for calcium carbonate and indicates it's less desirable to take considering Jessica's age**
 - Calcium carbonate requires stomach acid for proper absorption
 - Stomach acid decreases with age
- **OpenFDA found no clinically significant interactions between vitamin D3 and her prescribed celecoxib medication**
- **OpenFDA found side effects of Celebrex may range from mild stomach discomfort to serious heart & digestive risks**
- **Integration of evidence found in PubMed and OpenFDA indicates taking calcium carbonate is not recommended due to Jessica's prescribed Celebrex**
- **Supplement Advisor ranked vitamin D3 higher than calcium carbonate due to her health profile**
- **Combined evidence of various servers (PubMed, OpenFDA, Supplement Advisor) resulted in a higher overall recommendation score for vitamin D3**

Usage scenario 1 (Ken): Creatine Monohydrate > BCAAs

Candidate Answers:

- **Supplement Advisor MCP ranks creatine monohydrate higher than BCAAs due to it satisfying multiple of Ken's health goals in his profile (building lean muscle, bone strength, and cellular support)**
- **PubMed identified evidence that strongly supports taking creatine monohydrate over BCAAs according to Ken's profile**
 - **Creatine monohydrate**
 - Builds muscle strength and recycles cellular energy
 - Directly satisfies most of Ken's health goals
 - **BCAAs**
 - Reduce exercise fatigue and muscle soreness. Not directly related to Ken's health goals
 - Unnecessary if user is taking adequate amounts of dietary proteins
 - Ken's Mediterranean diet is already moderately high in protein
- **Most creatine monohydrate supplements are cheaper than BCAA supplements, according to**

iHerb-CLI

- Satisfies Ken's planning constraints for finding affordable supplements
- **Combined evidence of various servers (PubMed, iHerb-CLI, Supplement Advisor) resulted in a higher overall recommendation score for creatine monohydrate**

Why it's hard without an ontology:

- Integration of various metrics (recency, authorship, accessibility, citation count, peer-review status, etc)
- Recommendation evidence is distributed across multiple MCP servers (PubMed,
- Prioritizing one metric over another
 - Open access > highly cited
- agent should provide an explainable justification for why it selected a paper with evidence supporting its recommendation.
- Provenance to record which MCP tools supplied each piece of evidence and how that evidence contributed to the selected paper.

5. Why were the MCP servers for a given orchestration invoked in this order?

Example query: provide me with supplement recommendations that suits my health profile

Usage Scenario 1 (Ken): Supplement Advisor => PubMed

Usage Scenario 2 (Jessica): Supplement Advisor => OpenFDA => PubMed

Candidate Answers:

- **Tool dependency: Both orchestrations call Supplement Advisor MCP first**
 - **Cannot check for evidence or screening information for a supplement without the supplement itself**
- **Prioritization of user-specified planning constraints**
 - **Ken: minimizing execution cost > exhaustive search**
 - **Is taking no medication and is not concerned with adverse effects as much as Jessica, therefore, his plan doesn't invoke OpenFDA**
 - **Jessica: maximize evidence quality, check medication interactions**
 - **Must call OpenFDA for adverse side effects or contraindications**
 - **Safety > evidence is probably a prioritization we should assume**

Why it's hard without an ontology:

- There are multiple criteria when selecting the first MCP server (data dependency, information coverage, domain authority, required output format, execution constraints)
- Execution logs and relational databases may tell you the order of MCP servers being called
 - This question asks for an explanation (difficult for relational databases)
 - Requires explicit semantics of an ontology

How to get the answer:

1. System extracts all information requirements implied by the user's request
2. System queries the ontology-grounded KG of candidate MCP servers that satisfy the information requirements of the user's query
3. System uses ontology to reason over server and tool dependencies to determine whether server B needs to be invoked before server A (ie. produces outputs required by other servers)
4. If multiple valid servers can be invoked first, the system uses the ontology to evaluate them according to modeled decision criteria (ie. domain authority, information coverage, execution cost, redundant tool invocations, etc)
5. The system chooses the best server to invoke first based on the decision criteria modeled by the ontology

6. Why was orchestration plan B selected over orchestration plan A? (mostly planning constraint-based)

Example query: provide me with supplement recommendations that suits my health profile

Usage scenario 1 (Ken) orchestration plan:

10. Get user profile
11. Get candidate supplements
 - a. Supplement_advisor: recommend_supplement(search_query) => supplement recommendations
12. Remove redundant supplements
13. Get supplement products
 - a. iHerb-CLI: search(supplement, sort=[price]) => list of supplement products
14. Rank supplement products by cost
15. Verify evidence
 - a. PubMed: search_articles(search_query) => PMIDs
 - b. PubMed: download_articles_batch(PMIDs) => article abstracts
 - c. summarize_evidence()
16. Rank supplement products by evidence
17. Select supplement product
18. Produce recommendations

Usage scenario 2 (Jessica) orchestration plan:

11. Get user profile
12. Get candidate supplements
 - a. Supplement_advisor: recommend_supplement() => supplement recommendations
13. Screen interactions
 - a. OpenFDA: search_drug_adverse_events(medication/supplement)
 - b. OpenFDA: search_drug_labels(supplement)
 - c. OpenFDA: search_drug_recalls(supplement)
14. Rank by screening
15. Verify evidence
 - a. PubMed: search_articles(search_query) => PMIDs
 - b. PubMed: download_articles_batch(PMIDs) => article abstracts
 - c. summarize_evidence()
16. Rank by evidence
17. Get supplement products
 - a. iHerb-CLI: search(supplement, sort=[rating]) => list of supplement products
18. Rank supplement products by rating
19. Select supplement product
20. Produce recommendations

Candidate Answers:

- **Ken: minimizes execution cost**
 - **Less tool calls than Jessica's**
 - **Is already healthy and is taking no medications, so he skips the screening (OpenFDA MCP)**
- **Ken: cost efficiency > exhaustive analysis**
 - **Removes redundant supplements and ranks by cost first before checking evidence**
 - **Minimizing search scope for literature retrieval**
- **Jessica: safety check > evidence-based > cost efficiency**
 - **Performs screening interactions before evidence-based retrieval because she wants to minimize risks of adverse effects of the supplements**
 - **Doesn't mind the cost so her recommendations don't rank by cost**
 - **Must call OpenFDA since she's on medication**
- **Jessica: performs comprehensive analysis**
 - **Prioritizes high-quality evidence and safety so the Recommender invokes tools from OpenFDA and PubMed**
 - **Invokes MPC of high domain authority**
 - **Doesn't try to minimize the search scope**

- **Rankings of Ken and Jessica's supplements are different**
 - **Ken: cost, evidence**
 - **Jessica: safety, evidence**
- **Different user-specified planning constraints (preferences of the user)**
 - **Ken: minimizing execution cost, recommending affordable supplements, avoiding unnecessary and redundant supplements**
 - **Jessica: maximize evidence quality, check medication interactions, prioritize supplement quality over price**
- **Different information requirements (information supplied to support recommendation)**
 - **Ken is young, has no medical conditions, and is not currently taking any medication**
 - **Calling OpenFDA is unnecessary for him since he's not taking any other medication**
 - **Jessica is older, diagnosed with osteoarthritis, and is taking medication**
 - **Safety screening is required. More emphasis on invoking OpenFDA and PubMed MCPs for safety and evidence-based retrieval**

Why it's hard without an ontology:

- Multiple criteria in choosing the most optimal orchestration plan (information requirements, **tool capabilities**, domain authority, dependency constraints, execution cost, and user constraints)
- Can traverse to find multiple candidate orchestration plans that satisfy the user's request
 - difficult to systematically compare and rank them without explicit semantic representations
- Explanations require traceable relationships between requirements, tool capabilities, planning objectives, and constraints.

How to get the answer:

1. System traverses through the ontology-grounded KG of available **MCP servers, tools, inputs, outputs**, and dependency relationships to retrieve all candidate **orchestration plans** that satisfy the **information requirements** of the **user's query**.
2. System uses the ontology to reason over discovered candidate orchestration plans and determine which plans satisfy all semantic constraints (ie. tool dependencies, required **information coverage**, **user constraints**, etc)
3. If multiple valid plans exist, the system uses the ontology to evaluate them according to modeled decision criteria (ie. domain authority, information coverage, execution cost, redundant tool invocations, etc)
4. System chooses the preferred orchestration plan based on the decision criteria modeled by the ontology
5. System records the preferred orchestration plan, the decision criteria supporting its selection, and all alternative orchestration plans as instances in the KG for provenance-awareness

7. What skills are required to satisfy the user's request?

Example query: Provide me with supplement recommendations that suit my health profile.

Usage scenario 1 (Ken):

- **Personalized Supplement Recommendation Skill**
- **Evidence Verification Skill**
- **Product Selection Skill**

Usage scenario 2 (Jessica):

- **Personalized Supplement Recommendation Skill**
- **Supplement Safety Screening Skill (due to her current medication and diagnosed health condition)**
- **Evidence Verification Skill**
- **Product Selection Skill**

How to get the answer:

1. System processes the user query to extract InformationRequirements to satisfy the user's request
2. System processes the user's health profile and preferred execution settings to determine constraints and additional InformationRequirements the user's OrchestrationPlan must follow (ex. Jessica's current medication and diagnosed health condition means the OrchestrationPlan must include safety screening for contraindications)
3. System uses the KG to map what Capabilities satisfy the InformationRequirements and the Skills that perform those Capabilities.
4. System returns the Skills required to satisfy the user's request

8. What skills depend on other skills?

The Evidence Verification Skill, Supplement Safety Screening Skill, and Product Selection Skill all depend the Personalized Supplement Recommendation Skill in order to execute. All of those skills require a CandidateSupplement as input, which is only returned by the Personalized Supplement Recommendation Skill as output.

How to get the answer:

1. System traverses the ontology-grounded KG and identifies all available Skills
2. The system retrieved produced and consumed InformationArtifacts for each Skill. Ex.
 - a. Personalized Supplement Recommendation Skill → produces CandidateSupplement
 - b. Evidence Verification Skill → consumes CandidateSupplement, produces EvidenceAssessment
 - c. Safety Screening Skill → consumes CandidateSupplement, produces SafetyAssessment
 - d. Product Selection Skill → consumes EvidenceAssessment, SafetyAssessment, and ProductList
3. Ontology reasons over semantic relationships between Skills and InformationArtifacts to infer skill dependencies. Skill B is dependent on Skill A if B consumes an Information Artifact produced by A.
4. The system constructs a Skill dependency graph of inferred relationships
5. The system returns the set of Skills that must be executed sequentially from the dependency graph.

9. Are all the prerequisites satisfied before executing this Skill given the agent's current phase in orchestration?

Example: about to execute Evidence Verification Skill when:

- HealthProfile ✓
- CandidateSupplement ✗
- SearchQuery ✓

No, the agent cannot execute the Evidence Verification Skill because not all the prerequisites have been satisfied (namely CandidateSupplement). The agent must call the Personalized Supplement Recommendation Skill, which returns a CandidateSupplement before executing the Evidence Verification Skill.

How to get the answer:

1. System retrieves the agent's its current phase in the orchestration plan and identifies the Skill it's trying to execute.
2. System retrieves the Skill's prerequisites by querying the Skill dependency graph mentioned in CQ 8.
3. System checks off which prerequisites have been satisfied based on the agent's current orchestration state. Prerequisites are InformationArtifacts that are required as input for the Skill
4. If any unsatisfied prerequisite InformationArtifacts are detected, the system traces those artifacts to their producing Skills
5. The system returns one of the following two states:
 - a. Executable: all prerequisites are satisfied
 - b. Blocked: one or more prerequisites are missing but can be satisfied by calling another

Skill

High level classes

- Agent
- User Query
- MCP Tool
- MCP Server
- Orchestration Plan
 - selectedPlan: Boolean
- Information Requirement
- Constraint
- Tool Capability
- Tool Invocation
- Information Artifact
- Decision Criterion
- Execution Metric
- Selection Decision
- Explanation
- User Profile
- Supplement
- Medication
- Health Goal
- Evidence
- Skill
- Skill Invocation

Supplement

- Properties
 - Ingredient
 - Dosage
 - Intended Benefit
 - Side Effect
 - Contraindication
 - Cost
 - Quality Rating

Medication

- Properties
 - Drug Name
 - Dosage
 - Interaction

Health Goal

- Muscle Growth
- Bone Health
- Heart Health
- Longevity
- Immune Health
- Recovery

Evidence

- Clinical Trial
- Systematic Review
- Meta-analysis
- Randomized Controlled Trial
- Guideline

Decision Criterion

- Information Coverage
- Domain Authority
- Execution Cost
- Safety
- Product Quality
- Product Cost
- Evidence Quality

Constraint

- User Constraint
 - Budget
 - Preferred Brands
 - Dietary Preference
 - Allergies
 - Medication
 - Evidence Preference
 - Quality Preference
 - Health Goals
- Planning Constraint
 - Minimize Tool Calls
 - Minimize Token Usage
 - Maximize Evidence Quality
 - Prioritize Safety
 - Minimize Execution Time

Information Artifact

- User Profile
- Medication List
- Supplement
- Supplement Information
- Product Listing
- Product Review
- Clinical Study
- Systematic Review
- Meta-analysis
- Drug Interaction Report
- Product Price
- Customer Rating
- Ingredient List

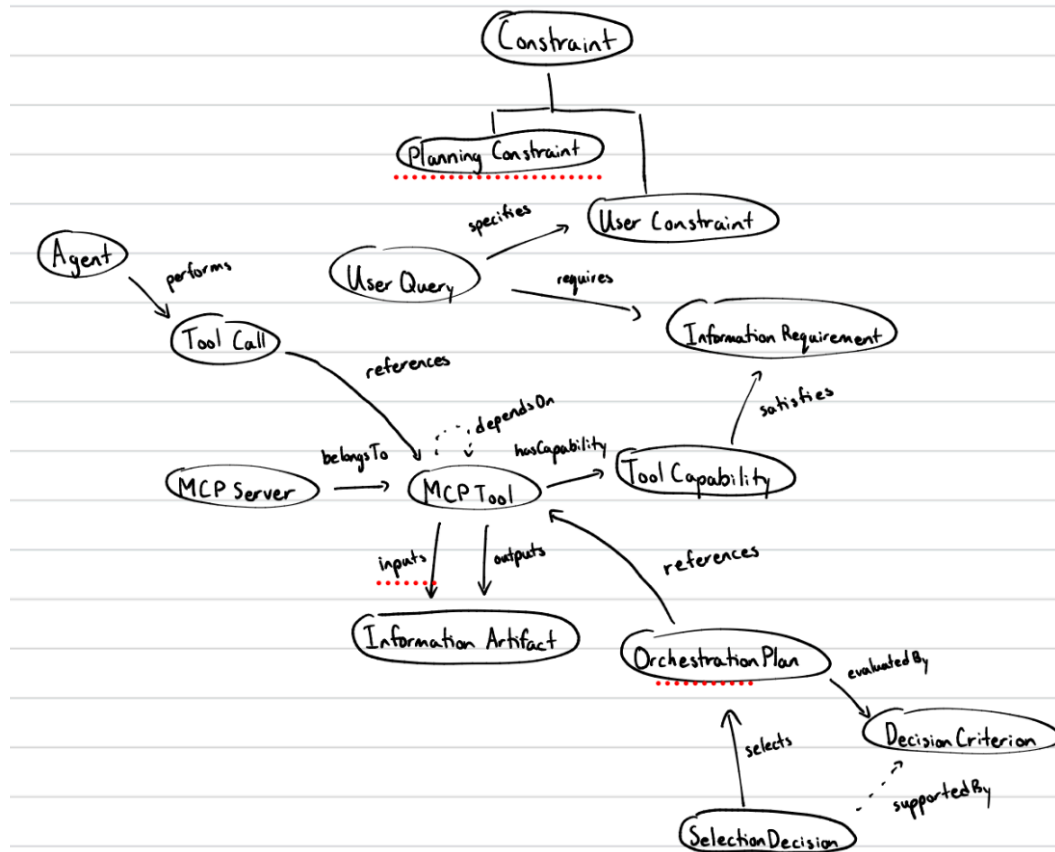
Tool Capability

- Retrieve User Profile
- Recommend Supplements
- Search Scientific Evidence
- Check Drug Interaction
- Retrieve Product Information
- Compare Prices
- Retrieve Customer Reviews
- Rank Supplements
- Filter Supplements
- Detect Redundant Supplements

Information Requirement

- User Profile Information

- Demographic
 - Sex
 - Age
 - Lifestyle
 - Activity
 - Activity Level
 - Dietary
 - Diet
 - Health Goal
 - Medication
 - Medical Condition
- Supplement Information
 - Candidate Supplement
 - Supplement Benefit
 - Supplement Dosage
 - Side Effect
 - Contraindication
- Evidence Information
 - Clinical Evidence
 - Strength
 - Guideline Recommendation
- Safety Information
 - Drug-Supplement Interaction
 - Supplement-Supplement Interaction
 - Adverse Effect
 - Drug Recall
 - Supplement Recall
- Product Information
 - Product Rating
 - Third-Party Testing
 - Customer Review
 - Product Availability
- Cost Information
 - Product Price



VIII. Resources

In order to support the capabilities described in this Use Case, a set of resources must be available and/or configured. These resources include the set of actors listed above, with additional detail, and any other ancillary systems, sensors, or services that are relevant to the problem/use case.

Knowledge Bases, Repositories, or other Data Sources

Data	Type	Characteristics	Description	Owner	Source	Access Policies & Usage
<i>(dataset or repository name)</i>	<i>(remote, local/in situ, etc.)</i>	<i>e.g. – no cloud cover</i>	<i>Short description of the dataset, possibly including rationale of the usage characteristics</i>		<i>Source (possibly a system, or remote site) for discovery and access</i>	
PubMed MCP Server	remote	Third-party MCP server	Connects to research repository for		https://github.com/aeghnnsw/pubmed-mcp	Open-source github

			retrieving biomedical-related papers			
Supplement Advisor MCP Server	remote	Third-party MCP server	Recommends supplements based on user's needs		https://github.com/erinheit451/supplement-advisor-mcp	Open-source github
OpenFDA MCP Server	remote	Third-party MCP server	Provides safety and regulatory information on FDA-regulated products (recalls, adverse effects, shortages)		https://github.com/Augmented-Nature/OpenFDA-MCP-Server	Open-source github
iHerb-CLI	remote	Third-party Client	Provides pricing and reviews for supplements from iHerb		https://github.com/SeverinAlexB/iherb-cli	Open-source github

External Ontologies, Vocabularies, or other Model Services

Resource	Language	Description	Owner	Source	Describes/Uses	Access Policies & Usage
<i>(ontology, vocabulary, or model name)</i>	<i>(ontology language and syntactic form, e.g., RDFS - N3)</i>	<i>If the service is one that runs a given ontology or model-based application at a given frequency, state that in addition to the basic description</i>		<i>Source (link to the registry or directly to the ontology, vocabulary, or model where that model is maintained, if available)</i>	<i>List of one or more data sources described by and/or used by the model</i>	
Representing Agentic Tools in Knowledge Graphs	Paper that proposes high-level ontology (core classes,			https://openreview.net/forum?id=7MV0H9Y3mi&noteId=LGFmuSRiu6		

for Structure-Aware Tool Discovery Under Tool Overload	relationships, taxonomies) No access to github or owl/rdf file					
AgentO	Ontology file (RDF/OWL)	Formalizes core concepts, roles, and relationships necessary to describe contemporary agent-based and workflow-oriented AI systems, with particular emphasis on LLM-driven agent frameworks	Kabul Kurniawan, Fajar J. Ekaputra	https://sepses.ifs.tuwien.ac.at/ontology-en.html	Use cases: Workflow reconstruction Workflow reuse Workflow auditing Captured classes: Agent, Task, Tool, Goal, Role, Capability, Workflow Pattern, Workflow Step	open-source

Other Resources, Service, or Triggers (e.g., event notification services, application services, etc.)

Resource	Type	Description	Owner	Source	Access Policies & Usage
<i>(sensor or external service name)</i>		<i>Include a description of the resource as well as availability, if applicable</i>	<i>Primary owner of the service</i>	Application or service URL ; if subscription based, include subscription and any subscription owner	
Open WebUI		<i>Web interface to connect AI models Supports MCP and Agent Skills</i>		https://knowledge.exlibrisgroup.com/University/Products/Documentation/Configuring/UsingUlrichsweb_API/Ulrichsweb%3A_Using_the_Ulrichsweb_API	<i>Docker image Runs locally</i>

--	--	--	--	--	--

IX. References and Bibliography

List all reference documents – policy documents, regulations, standards, de-facto standards, glossaries, dictionaries and thesauri, taxonomies, and any other reference materials considered relevant to the use case

X. Notes

There is always some piece of information that is required that has no other place to go. This is the place for that information.